

IA Aplicada com LLMs

Aula 07 — O pipeline, e os quatro modos de falha

O pipeline é da Aula 06

pergunta → [busca] → portão → [contexto + LLM] → resposta

o nome	RAG (LEWIS et al., 2020)
a classificação	<i>prompt chaining</i> com portão — não é um agente
o índice	banco de vetores, por persistência

Montado na [nota 04 de lá](#). Aqui ele entra pronto.

E ele não confere nada

O portão daquele laboratório tem limiar de **0,6 de distância**.

```
'reajuste da tabela de fretes marítimos?'    d = 0,2586    <- passou
```

A pergunta não tem resposta no corpus, e o modelo respondeu assim mesmo. **É neste ponto que esta aula entra.**

Os quatro modos de falha

Os quatro não são da mesma natureza

Modo	Natureza	A causa já tem nome
1 recuperou errado	recuperação	cegueira de negação (Aula 06)
2 ignorou o certo	geração	— o trecho estava lá
3 inventou	geração	— não havia o que recuperar
4 contraditórios	recuperação	número + tempo (Aula 06)

falha de RECUPERAÇÃO -> conserta no índice, no corte, no k
falha de GERAÇÃO -> conserta no prompt e no contrato de saída

O 1 e o 4 já existiam antes desta aula

A Aula 06 os demonstrou em **cos seno puro** — inclusive num router, onde o erro acontece sem nenhum modelo escrever uma palavra.

O RAG não cria essas falhas: cria a plausibilidade delas. O trecho errado continua errado, mas agora vem embrulhado numa resposta bem redigida, **com fonte citada.**

Trocar um diagnóstico pelo outro é o que faz alguém passar uma semana ajustando *chunking* para consertar um defeito que está no *prompt*.

Modo 1 — recuperou o trecho errado

```
pergunta: "posso reembolsar almoço em viagem internacional?"  
recuperou: Art. 4º §1º (limite nacional, R$ 120,00)  
esperado: Art. 4º §2º (viagem internacional, R$ 260,00)
```

A resposta sai **fundamentada e errada**. O modelo fez o trabalho corretamente sobre o insumo errado.

Sintoma: `recall@k` baixo. Onde mexer: **índice** — corte, modelo de embedding, ou `k`.

Modo 2 — recuperou o certo e ignorou

O trecho correto **estava no contexto**, e a resposta não o usa.

Causa típica: *lost in the middle* (Aula 02, nota 01, §4.3). O trecho decisivo ocupava a posição 3 de 5.

ordem A:	[certo, ruído, ruído]	-> resposta correta
ordem B:	[ruído, certo, ruído]	-> resposta incorreta

Reordenar os trechos muda a resposta, com o mesmo conteúdo recuperado. É a demonstração mais desconfortável da aula.

Modo 3 — não havia resposta, e o modelo inventou

```
pergunta: "qual o procedimento para férias coletivas?"  
corpus:   regulamento de despesas – não trata do assunto  
resposta: "O procedimento exige aviso prévio de 15 dias..."
```

O modelo é **treinado para responder**. Sem uma rota explícita de recusa, ele produz a resposta mais plausível.

A recusa não é uma falha do sistema. É uma **função** que precisa existir — e a nota 05 a constrói.

Modo 4 — trechos contraditórios

recuperado 1:	"Art. 4º §1º – limite de R\$ 120,00"	(vigente)
recuperado 2:	"Art. 4º §1º – limite de R\$ 90,00"	(REVOGADO)

Os dois estão no corpus. Os dois são igualmente similares à pergunta — porque o vetor **não tem noção de anterioridade** (a cegueira de tempo, Aula 06).

E aqui as duas métricas da nota 03 **aprovam** a execução, com a resposta errada. **O defeito é do corpus, e não se conserta com recuperação.**

Os quatro, resumidos

#	Falha	Sintoma	Onde mexer
1	recuperou o errado	recall baixo	índice
2	recuperou e ignorou	recall alto, fidelidade baixa	prompt
3	inventou	resposta sem lastro	contrato de saída
4	contradiisse	ambas as métricas altas	corpus

A quarta linha é a que quase ninguém considera, e é a mais desconfortável: o sistema fez tudo certo sobre um documento desatualizado.

O que fica

- RAG é **prompt chaining com portão** — não é agente
- O índice persiste e **filtra por metadado**, que é a defesa das cegueiras
- Quatro modos de falha, e **cada um tem um endereço diferente**
- A ordem dos trechos muda a resposta

Próxima nota: o contrato de saída que torna a falha 1 **inspecionável** e a falha 3 **declarável**.